BLOCKCHAIN TECHNOLOGIES MINI-PROJECT

on

AI Prompt MarketPlace – Blockchain-Based Platform

for Buying and Selling AI Prompts

Submitted By

Rahul Aitla (01)

Chetan Banjan (05)

Maitreya Dongardive (20)

Under the Guidance of

Dr. Gauri Deshpande

Department of Computer Science & Engineering (Data Science)

Academic Year :- 2026-27

INDEX

SR. NO.	CHAPTER NAME	PAGE NO.
1	Introduction	1
2	Problem Statement	1
3	Objectives	1
4	Technologies Used	2
5	System Architecture	2
6	Methodology	2-3
7	Smart Contract	4
8	Testing	4
9	Result	5-8
10	Advantages	8
11	Limitations	8
12	Future Scope	8

13	Conclusion	9
14	References	9

1. Introduction

Blockchain is a distributed ledger technology that records transactions in a sequence of blocks. Once transactions are confirmed, they become difficult to alter, providing transparency and security. Smart contracts are programs deployed on a blockchain that automatically execute predefined operations when specified conditions are met.

In this project, AI Prompt MarketPlace is a blockchain-based marketplace where sellers publish AI prompt listings and buyers purchase them using ETH. A Solidity smart contract records each listing on-chain and handles exact-price payments, transferring funds directly to the seller's wallet. The platform supports wallet-based sign-in using MetaMask, a live playground where buyers can try prompts before purchasing, purchase-gated file downloads through signed links, and buyer reviews. This improves trust, transparency and payment security between prompt sellers and buyers.

2. Problem Statement

Traditional platforms for selling AI prompts depend on centralized payment processors, give buyers no reliable way to check prompt quality before paying, and provide little transparency into transactions. A blockchain-based marketplace can secure payments through a smart contract, record listings and purchases on-chain, and let buyers verify purchases directly against the blockchain.

3. Objectives

- To understand the basic concept of blockchain.
- To develop a smart contract for marketplace listings and payments.
- To record prompt listings and purchases on the blockchain.
- To connect a web application to the contract through a crypto wallet.
- To demonstrate a practical application of smart contracts.

4. Technologies Used

Technology	Purpose
Solidity	Smart contract development
Hardhat	Smart contract testing and deployment
Ethereum (Hardhat / Sepolia)	Blockchain platform
MetaMask	Wallet connection and transactions
Next.js, React, TypeScript	Frontend and backend framework
PostgreSQL + Prisma	Database and ORM
wagmi, viem, RainbowKit	Wallet connection and contract interaction
SIWE + JWT	Wallet-based authentication
Groq API	Live prompt playground LLM

5. System Architecture

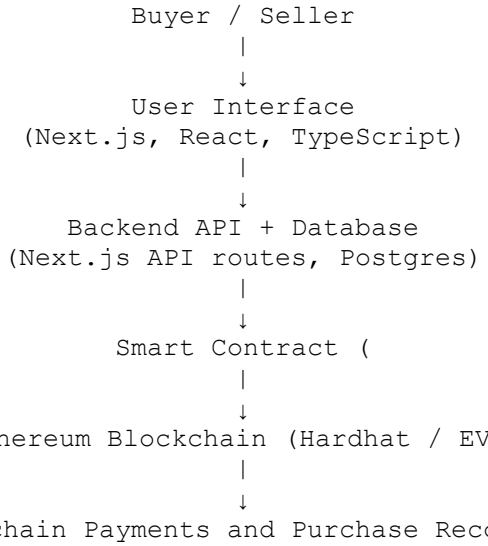


Fig 5.1: AI Prompt MarketPlace System Architecture

6. Methodology

Step 1

Design a Solidity smart contract for listing and purchasing AI prompt assets.

Step 2

Define marketplace data such as:

- Asset ID and seller wallet address
- Listing price in wei
- Purchase status per buyer

Step 3

Implement listAsset() and purchase() functions with exact-payment verification and event emission.

Step 4

Deploy the smart contract on a local Ethereum blockchain using Hardhat.

Step 5

Connect the Next.js frontend to the contract using wagmi, viem, RainbowKit and MetaMask.

Step 6

Allow sellers to publish prompt listings on-chain and buyers to purchase them with ETH; gate prompt file downloads with signed links.

Step 7

Run the Hardhat test suite to verify listing validation, exact-payment purchase, and duplicate-purchase protection.

Implementation and Results

7. Smart Contract

A simple contract can contain:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract Marketplace is ReentrancyGuard {
    mapping(uint256 => address payable) public assetSeller;
    mapping(uint256 => uint256) public assetPrice;
    mapping(uint256 => mapping(address => bool)) public hasPurchased;

    event AssetListed(uint256 indexed assetId, address indexed seller,
uint256 price);
    event Purchased(uint256 indexed assetId, address indexed buyer, address
indexed seller, uint256 amount, uint256 timestamp);

    function listAsset(uint256 assetId, uint256 priceWei) external {
        require(assetId != 0, "assetId 0 is reserved");
        require(assetSeller[assetId] == address(0), "Already listed");
        require(priceWei > 0, "Price must be > 0");
        assetSeller[assetId] = payable(msg.sender);
        assetPrice[assetId] = priceWei;
        emit AssetListed(assetId, msg.sender, priceWei);
    }

    function purchase(uint256 assetId) external payable nonReentrant {
        address payable seller = assetSeller[assetId];
        require(seller != address(0), "Not listed");
        require(msg.value == assetPrice[assetId], "Incorrect payment
amount");
        require(!hasPurchased[assetId][msg.sender], "Already purchased");

        hasPurchased[assetId][msg.sender] = true;

        (bool sent, ) = seller.call{value: msg.value}("");
        require(sent, "Transfer to seller failed");

        emit Purchased(assetId, msg.sender, seller, msg.value,
block.timestamp);
    }
}
```

8. Testing

Test Case	Input	Expected Result	Status

Test Case	Input	Expected Result	Status
TC01	List asset with valid id and price	AssetListed event emitted	Pass
TC02	List asset with assetId = 0	Transaction reverts	Pass
TC03	List the same assetId twice	Transaction reverts	Pass
TC04	List asset with price 0	Transaction reverts	Pass
TC05	Purchase below listed price	Transaction reverts	Pass
TC06	Purchase above listed price	Transaction reverts	Pass
TC07	Purchase with exact price	ETH sent to seller, Purchased event emitted	Pass
TC08	Same buyer purchases twice	Transaction reverts	Pass
TC09	Purchase unlisted assetId	Transaction reverts	Pass

9. Result

The AI Prompt MarketPlace platform was successfully implemented and tested on a local Ethereum blockchain using Hardhat. Sellers can publish prompt listings on-chain, and buyers can connect a wallet with MetaMask, try prompts in a live playground, purchase listings with exact ETH payments, and download the prompt file through a signed link. All nine smart contract test cases passed.

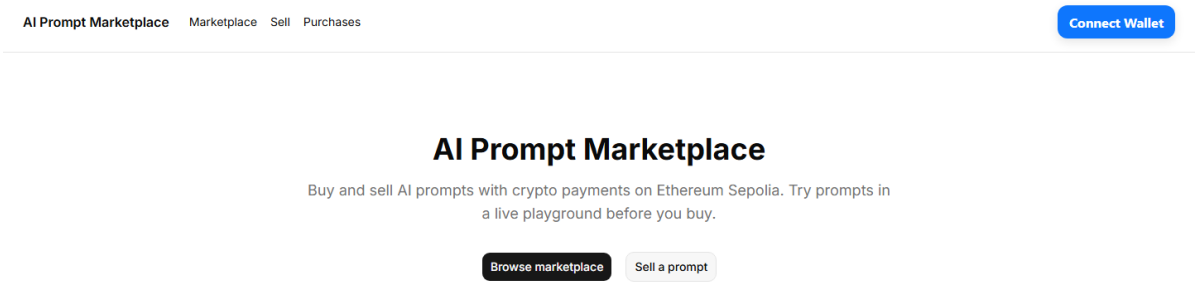


Fig 9.1: AI Prompt MarketPlace Landing Page

The landing page introduces the AI Prompt MarketPlace and highlights its main purpose: buying and selling AI prompts with ETH payments secured by a smart contract.

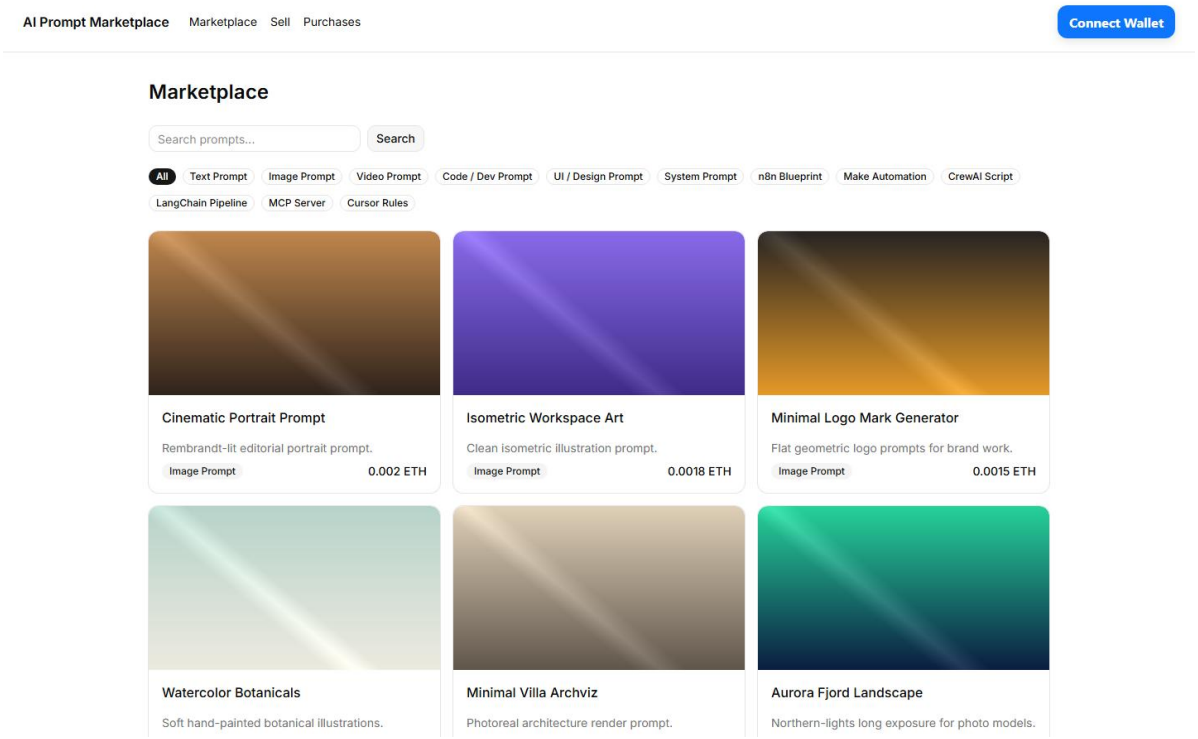


Fig 9.2: Marketplace Catalog with Prompt Listings

The catalog displays published prompt listings with category filters, text search, and prices denominated in ETH.

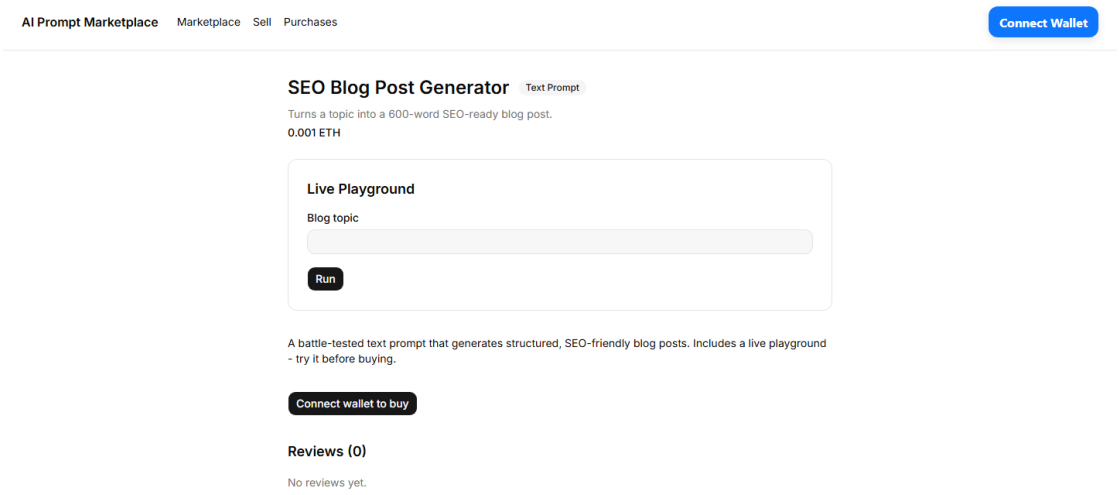


Fig 9.3: Asset Detail Page with Live Playground

Buyers can view the prompt details and try the prompt in a live LLM playground before deciding to purchase it.

```
=== AssetListed ===
tx hash : 0xf1984d949daebed1380157fe6eb5ed44a5386ad41771fa2a83ce039d826664fd
block   : 28
gas     : 68644
seller  : 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
price   : 0.002 ETH
=== Purchased ===
tx hash : 0x457f8881cb118b28568be4db22cd7d2985c80e63fe04fab1fc5d4c3f5b4dffd5
block   : 29
gas     : 63818
buyer   : 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
seller  : 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
amount  : 0.002 ETH transferred to seller
hasPurchased(buyer) : true
```

Fig 9.4: On-chain Listing and Purchase Transactions

The AssetListed and Purchased events record the listing and the payment on the blockchain, along with transaction hashes, block numbers and gas used.

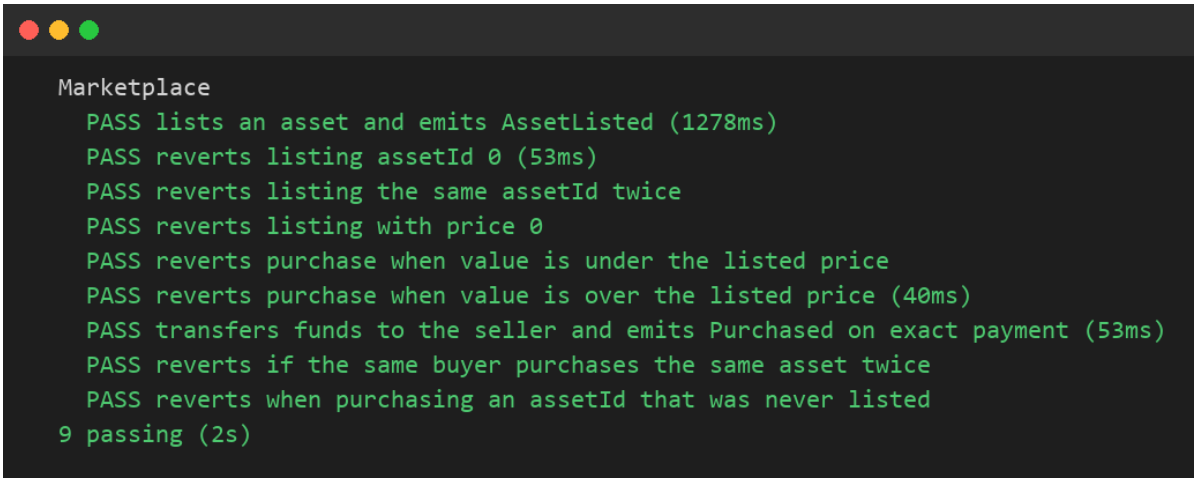


Fig 9.5: Smart Contract Test Results (Hardhat)

All nine test cases pass, covering listing validation, exact-payment purchase, and duplicate-purchase protection.

10. Advantages

- Secures payments using a smart contract with exact-price enforcement.
- Improves transparency – listings and purchases are recorded on-chain.
- Payments go directly to the seller with no intermediary.
- Buyers can try prompts in a live playground before paying.
- Demonstrates practical use of blockchain technology.

11. Limitations

- Requires a crypto wallet (MetaMask) and blockchain access.
- Blockchain transactions are difficult to reverse once confirmed.
- Transaction (gas) fees apply on public blockchains.
- The project is a prototype running on a local/test network, not a full production marketplace.

12. Future Scope

The system can be extended by adding:

- Deploy the contract on a public testnet or mainnet
- Buyer ratings and on-chain reviews
- Dispute resolution and refund mechanism
- IPFS or decentralized storage for prompt files

- NFT-based licenses for prompt ownership
- Support for more prompt categories and collections
- Mobile-friendly interface and purchase notifications

13. Conclusion

The project demonstrates the use of blockchain and smart contracts for a decentralized AI prompt marketplace. Listings and purchases are managed by a smart contract, payments are transferred directly to sellers in ETH, and prompt downloads are gated by verified on-chain purchases. Testing on a local blockchain successfully demonstrated the listing, purchase and payment flow.

14. References

1. https://ethereum.org/developers/docs/
2. https://docs.soliditylang.org/
3. https://hardhat.org/docs
4. https://nextjs.org/docs
5. https://docs.openzeppelin.com/contracts